

数位DP入门

主讲：黄凤林



➔ 数位dp基础知识

介绍：

数位dp是一种**计数用的dp**，一般就是要**统计一个区间 $[L,R]$ 内满足一些条件数的个数**。

所谓数位dp，字面意思就是在数位上进行dp（也称按位dp）。

数位的含义：一个数有个位、十位、百位、千位……数的每一位就是数位啦！

数位dp的实质就是**换一种暴力枚举的方式**，使得新的**枚举方式满足dp的性质**，然后记忆化实现。

➔ 数位dp基础知识

应用举例：比如求 $[L,R]$ 内满足条件数的个数

①普通暴力枚举如下：

```
for(int i=L;i<=R;i++) if(judge(i)) ans++;
```

然而**这样枚举不方便记忆化，或者说根本无状态可言。**

①新的枚举：控制上界枚举，从最高位开始向下枚举
例如： $ri=313$ ，那么我们从百位开始枚举：百位可能的情况有0,1,2,3。这样子枚举的优势在于有状态可言，可记忆化。比如百位枚举2，后面十位和个位的枚举范围都是0--9；百位枚举1，后面十位和个位的枚举范围依然是0--9，我们发现此时的十位和个位取值完全是一样的，也就是这两位方案数完全相同，所以我们就可以记录下来保存，以避免后面重复计算。



➔ 数位dp一般思路

① 统计 $[L, R]$ 的方案数:

通过统计 $[1, R]$ 和 $[1, L-1]$ 的数量，再相减就是 $[L, R]$ 的数量

② 新枚举方法的思路: 控制上界枚举，从最高位开始往下枚举

每一位枚举都不能让枚举的这个数超过上界313（下界就是0、1、2，这个次要），当百位枚举了1，那么十位枚举就是从0到9，因为百位1已经比上界3小了，后面数位枚举什么都不可能超过上界。

问题就在于：当高位枚举**刚好达到上界**时，那么**紧接着的一位枚举就有上界限制**。具体的这里如果百位枚举了3，那么十位的枚举情况就是0到1，如果前两位枚举了31，最后一位只能是0到3（这一点一般需要用**一个变量专门来判断枚举范围**）。

最后一个问题：最高位枚举0：百位枚举0，相当于此时我枚举的这个数最多是两位数，如果十位继续枚举0，那么我枚举的就是个位数，因为我们要枚举的是小于等于R的所以数，当然不能少了位数比R小的咯！（这样枚举是为了无遗漏的枚举，不过可能会带来一个问题，就是前导零的问题，不过这个不是每个题目都是会有影响的，可能前导零不会影响我们计数，具体要看题目具体分析）



→ 枚举模板

dp思想：枚举到当前位 pos ，状态为 $state$ (这个就是根据题目来的，可能很多，毕竟dp千变万化)的数量(既然是计数,dp值显然是保存满足条件数的个数)

比如求：数位上不能出现连续的两个1(11、112、211都是不合法的)合法数的个数。

问题分析：状态 $dp[pos][pre]$:表示当前枚举到 pos 位，前面一位枚举的值为 pre (更加前面的位已经合法了)的方案数(我的 pos 从0开始)

➔ 数位dp一般实现

```
int dfs(int pos,int pre,int lim)//lim表示枚举上界
{
    if(pos==-1) return 1;//递归边界
    if(!lim && dp[pos][pre]!=-1) return dp[pos][pre];
    int up=lim?a[pos]:9;//up表示当前位值可枚举范围上界
    int temp =0;
    for(int i=0;i<=up;i++){//枚举当前位可取的值
        if(i==1 && pre==1) continue;//不合法的剔除
        temp+=dfs(pos-1,i,lim && i==a[pos]); //高位往低位枚举
    }
    if(!lim) dp[pos][pre]=temp;//lim为真仅当所有位都取最高位的值
    return temp;
}
```

```
int work(int x){
    int pos=0;
    while(x){
        a[pos++]=x%10; x/=10;
    }
    return dfs(pos-1,-1,true);
}
```

```
scanf("%d%d",&L,&R);
memset(dp,-1,sizeof(dp));
int ans1=work(L-1);
int ans2=work(R);
printf("%d\n",ans2-ans1);
```



→ 实战篇

例1: HDU 2089 不要62

问题描述:

杭州人称那些傻乎乎粘嗒嗒的人为62（音：laoer）。

杭州交通管理局经常会扩充一些的士车牌照，新近出来一个好消息，以后上牌照，不再含有不吉利的数字了，这样一来，就可以消除个别的士司机和乘客的心理障碍，更安全地服务大众。

不吉利的数字为所有含有4或62的号码。例如：

6215 734 884都属于不吉利号码。但是，61152虽然含有6和2，但不是62连号，所以不属于不吉利数字之列。

你的任务是，对于每次给出的一个牌照区间号，推断出交管局今次又要实际上给多少辆新的士车上牌照。



→ 问题分析

题意：就是数位上不能有4也不能有连续的62

方法：

针对没有4的话在枚举的时候判断一下，不枚举4就可以保证状态合法了，所以这个约束没有记忆化的必要。

对于62的话，涉及到两位，需要判断前一位是6或者不是6这两种不同情况，计数是不相同的，所以要用状态来记录不同的方案数。

所以用 $dp[pos][re]$ 表示当前第 pos 位，前一位值为 pre 的方案数。



→ 参考代码

```
int dfs(int pos,int pre,int lim){
    if(pos==-1) return 1;
    if(!lim && dp[pos][pre]!=-1) return dp[pos][pre];
    int up=lim?a[pos]:9;//条件语句, 枚举上界
    int temp=0;
    for(int i=0;i<=up;i++){
        if(i==4) continue;
        if(i==2 && pre==6) continue;
        temp+=dfs(pos-1,i,lim && i==a[pos]);
    }
    if(!lim) dp[pos][pre]=temp;
    return temp;
}
```

```
int main(){
    memset(dp,-1,sizeof(dp));
    scanf("%d%d",&L,&R);
    while(L+R>0){
        printf("%d\n",solve(R)-solve(L-1));
        scanf("%d%d",&L,&R);
    }
    return 0;
}
```



→ 实战篇

例2：问题描述：求 n 位数里面能被 m 整除的数的个数
数据范围： $N \leq 10^{15}$ ， $m < 100000$

问题分析：

朴素方法：从最小的 n 位数枚举到最大的 n 位数，然后判断是否能被 m 整除即可。

时间复杂度 $O(n)$ ，**超时，不解释**！怎么办？

寻求新的解决方法，转变思路，直接枚举数字大小是不行的，那我按照 n 数字的位数，一位一位枚举，即数位DP。



➔ 问题分析

一般数位DP是多维的，首先会有一维表示的是当前到了第几位，通常情况这一维比较固定。其它的就是根据题目的实际要求了，如本题就需要新增一维来达到最后整除的效果。

做法：用一维来记录除以 m 的余数。

对于当前的余数 j ，在下一位加入了 k ，下一位的余数为多少？

根据同余，我们可以得到：余数变成了 $(j*10+k) \bmod m$ 。

设 $dp[i][j]$ 表示当前到了第 i 位，除以 m 的余数为 j 的方案数。

最后的答案是什么？

首先，第一维自然是 n ，

那么第二维取什么值呢？

显而易见，要求是的倍数，所以第二维是 0 。

综上所述，最后的答案是 $dp[n][0]$ 。

→ 参考代码1

```
void answer(){
    for(int i=1;i<=9;i++) dp[1][i%m]+=1;
    for(int i=2;i<=n;i++){//枚举pos当前位
        for(int h=0;h<m;h++){ //当前位的可能余数
            for(int k=0;k<=9;k++){ //枚举pos当前位可取数的范围
                for(int j=0;j<m;j++){ //枚举上一位的余数
                    if(h==(j*10+k)%m) dp[i][h]+=dp[i-1][j];
                }
            }
        }
    }
    for(int i=0;i<=m-1;i++){
        printf("%d %lld\n",i,dp[n][i]);
    }
}
```

效率任然较低，如何优化一下？

当前位的可能余数是由上一位的余数和当前位的取值唯一确定的，因此可以省掉当前可能余数枚举这从循环

→ 参考代码2

```
void work(){
    for(int i=1;i<=9;i++)dp[1][i%m]+=1;
    for(int i=1;i<n;i++){
        for(int k=0;k<=9;k++){
            for(int j=0;j<m;j++){
                dp[i+1][(j*10+k)%m]+=dp[i][j];
            }
        }
    }
    for(int i=0;i<=m-1;i++){
        printf("%d %lld\n",i,dp[n][i]);
    }
}
```

注意思考两种实现方式的差异、区别和重要性？

→ 参考代码3--dfs实现

```
long long dfs(int pos,int k)//dp[pos][k]表示枚举到当前pos位, 余数位k的方案数
{
    if(dp[pos][k]!=0 || pos==1) return dp[pos][k];
    for(int i=0;i<=9;i++)//枚举当前位pos位的取值范围
    {
        for(int j=0;j<m;j++)//枚举pos-1位的可能余数范围
        {
            int yu=(j*10+i)%m;
            if(k==yu) dp[pos][k]+=dfs(pos-1,j);
        }
    }
    return dp[pos][k];
}
```

```
memset(dp,0,sizeof(dp));
for(int i=1;i<=9;i++) dp[1][i%m]+=1;//边界预处理
dfs(n,0);
printf("--%d\n",dp[n][0]);
```



→ 实战篇

例3: [Hdu3652](http://acm.hdu.edu.cn/showproblem.php?pid=3652)

题目链接:

<http://acm.hdu.edu.cn/showproblem.php?pid=3652>

题目大意: 求小于 n 是13的倍数且含有'13'的数的个数。

问题分析: 本题就是要求含有13和被13整除数的个数, 因此我们可以用上题类似的方法

用 $dp[pos][k][sta]$ 表示到 pos 中余数为 k , 包含13的状态位 sta 的方案数, 具体:

$dp[pos][k][2]$ 表示非限制条件下到 pos 位模13余 k 且已含有13的总个数;

$dp[pos][mod][1]$ 表示没含有13但前一位是1且模13余 k 的总个数;

$dp[pos][mod][0]$ 表示没含有13前一位不为1模13余 k 的总个数

只是注意, 这里是小于 n 的数, 不在是 n 位数。

所以每一位有一个枚举上限



参考代码

```
int dfs(int pos,int pre,int md,int flag,int limit)
{
    if(!pos) return flag&&md==0;
    if(!limit&&flag&&~dp[pos][md][2]) return dp[pos][md][2];
    if(!limit&&!flag&&pre==1&&~dp[pos][md][1]) return dp[pos][md][1];
    if(!limit&&!flag&&pre!=1&&~dp[pos][md][0]) return dp[pos][md][0];
    int up=limit?dig[pos]:9;
    int ans=0;
    for(int i=0;i<=up;i++)
    {
        ans+=dfs(pos-1,i,(md*10+i)%13,flag||(pre==1&&i==3),limit&&i==up);
    }
    if(!limit)
    {
        if(flag)dp[pos][md][2]=ans;
        else if(pre==1)dp[pos][md][1]=ans;
        else dp[pos][md][0]=ans;
    }
    return ans;
}
```



→ 参考学习地址

我的dp之路

<https://www.cnblogs.com/lien-us/category/616874.html>

<https://www.cnblogs.com/lien-us/p/4248378.html>

hdu:3555: 给定x, 输出从0~x所有的数中, 含有49的所有数的个数。

hdu:F(X)给定两个数a,b。还有一个加权函数,

$$F(x) = A_n * 2^{n-1} + A_{n-1} * 2^{n-2} + \dots + A_2 * 2 + A_1 * 1。$$

然后求, 0~b的各个数中进行F(x)不大于F(a)的数字的个数。

Luogu 1831

Luogu 2602

Luogu 3413



→ 升级篇

例1: HDU 4734

问题描述:

题目给了个 $f(x)$ 的定义: $F(x) = A_n * 2^{n-1} + A_{n-1} * 2^{n-2} + \dots + A_2 * 2 + A_1 * 1$,
 A_i 是十进制数位, 然后给出 a, b 求区间 $[0, b]$ 内满足 $f(i) \leq f(a)$ 的 i 的个数。



→ 升级篇

例2: [ural1057](http://acm.hust.edu.cn/vjudge/problem/viewProblem.action?id=18851)

题目链接:

<http://acm.hust.edu.cn/vjudge/problem/viewProblem.action?id=18851>或

<http://acm.timus.ru/problem.aspx?space=1&num=1057>

题目大意: 求给定区间 $[X, Y]$ 中满足下列条件的整数个数: 这个数恰好等于 K 个互不相等的 B 的整数次幂之和。例如, 设 $X=15$, $Y=20$, $K=2$, $B=2$, 则有且仅有下列三个数满足题意:

$$17 = 2^4 + 2^0,$$

$$18 = 2^4 + 2^1,$$

$$20 = 2^4 + 2^2.$$

$$1 \leq X \leq Y \leq 2^{31}-1, \quad 1 \leq K \leq 20, \quad 2 \leq B \leq 10.$$



→ 升级篇

题目大意：

给定长度为 n 的序列 $A[i]$ ，求所有 $A[i] \text{ xor } A[j]$ ($i < j$)的值之和。



➔ 升级篇

启示录(POJ 3208)

题目大意:

定义十进制下有3个连续的6的数为魔鬼数。有T个询问, 求第X小的魔鬼数。

$X \leq 5 \times 10^7$, 每个测试点包含不超过1000组数据。

问题分析:

1、朴素方法, 就是从最小的三位数开始枚举, 然后判断该数是否是魔鬼数。

最后找到第x小的魔鬼数。时间复杂度为 $O(x \times 83 \times 1000)$ 。肯定是不行的。



➔ 启示录(POJ 3208)

- 用动态规划的思想来做，我们先确定第 x 小的魔鬼数的位数，确定位数后，在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。
- 首先思考如何确定第 x 小的魔鬼数的位数，直接做是没法做的，我们只能找出 i 位数字有多少个魔鬼数来枚举处理。

➔ 启示录(POJ 3208)

- ✓ 用 $F[i][3]$ 表示由 i 位数字构成的魔鬼数有多少个。
- ✓ $F[i][j](0 \leq j \leq 2)$ 表示由 i 位数字构成且开头已经连续有 j 个6的非魔鬼数有多少个。
- ✓ 注意，在计算 F 时，我们允许前导0的存在。
- ✓ 考虑第 i 位(最高位)是什么数字，容易得到递归转移方程：
- ✓ $F[i][0] = 9 * (F[i-1][0] + F[i-1][1] + F[i-1][2])$
- ✓ $F[i][1] = F[i-1][0]$ $F[i][2] = F[i][1]$
- ✓ $F[i][3] = F[i-1][2] + 10 * F[i-1][3]$

```
void Init(){
    f[0][0]=1;
    for(int i=0;i<=20;i++){
        for(int j=0;j<3;j++){
            f[i+1][j+1]+=f[i][j];
            f[i+1][0]+=f[i][j]*9;
        }
        f[i+1][3]+=f[i][3]*10;
    }
}
```



→ 启示录(POJ 3208)

通过 $F[i][3]$ 确定第 X 小的魔鬼数的位数，然后在用“试填”的方法，从左往右依次考虑每个数位。同时记录当前末尾已经有连续的几个6。

从小到大枚举当前位数填什么数字，通过预处理的 F 数组可以直接计算出后边的数位有多少种填法可以得到魔鬼数，与 X 比较即可，代码实现很具技巧性。

dp参考代码

```
while(f[m][3]<n) m++;
for(int i=m,k=0;i>0;i--){//试填第i位, 末尾已有K个6(k==3也表示已经是魔鬼数)
    for(int j=0;j<=9;j++){//从小到大枚举第i位填的数字
        long long cnt=f[i-1][3];//3个连续6全部来自于后i-1位
        if(j==6||k==3)//枚举的当前位是6或者已经有3个连续的6了
            for(int L=max(3-k-(j==6),0);L<3;L++)//妙用
                cnt+=f[i-1][L];
        if(cnt<n) n-=cnt;//如果cnt比n小, 说明第n个魔鬼数的第i位应该比j更大
        else//否则, 第i位就应该是j
        {
            if(k<3)
            {
                if(j==6) k++; else k=0;
            }
            printf("%d",j);
            break;
        }
    }
}
```



→ 月之谜(BZOJ 1799)

题目大意：

- 如果一个十进制数能够被它的各位数字之和整除，则称这个数为“月之数”。
- 给定整数L和R，你需要计算闭区间[L, R]中有多少个月之数
- $1 \leq L, R \leq 2^{31}$, 每个测试点包含不超过3000组数据。

The background features a central point from which several rays of different colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a line graph with two fluctuating lines (one blue, one orange) is visible on the right side.

Thank you!